

CSCE 625 ARTIFICIAL INTELLIGENCE · MODULE 13 · LECTURE NOTES

Choosing an Approach

And saying what the agent can do.

MODULE OUTCOMES

- Choose a method from a problem's properties.
- Explain where learned and symbolic methods each belong.
- Explain hybrid architectures and why they dominate.
- Evaluate an agent honestly.
- State what an AI system can be trusted to do.

1 Choosing a method

If the problem is	Use	Module
Find a route through space	A* on a navigation mesh, with a hierarchy if the level is large.	04, 03
Find a sequence of actions to achieve a goal	Classical planning if the actions are declarative; A* over states otherwise.	07, 02, 03
Satisfy a set of constraints, with no costs to minimise	A constraint solver, or encode to SAT.	06, 08
Beat an opponent	Alpha-beta if a good evaluation exists; MCTS if not.	05
Act under uncertainty you can model	An MDP, solved offline and shipped as a policy table.	10
Infer hidden state from observations	A Bayesian network, or a filter if it is sequential.	09
Produce plausible behaviour cheaply and controllably	A behaviour tree, or a utility system if concerns compete.	11
Unknown dynamics, and a great deal of interaction available	Reinforcement learning.	CSCE 642

Work down that table before reaching for a learned method. Most game and robotics problems are answered in the first seven rows — faster, more controllably, and with guarantees — and reaching for the eighth row first is the most common and most expensive error in applied AI.

TWO QUESTIONS THAT RESOLVE MOST CHOICES

Do you have a model of the dynamics? If yes, you can compute — which is this entire course. If no, either learn the policy from interaction (CSCE 642) or build a model and then compute, and **building a model is underrated**: an approximate model plus exact computation frequently beats no model plus learning, and it is debuggable.

Does a designer need to control the behaviour directly? If yes, use an authorable architecture (Module 11) and accept suboptimality. If no, optimality is available — **and 'no' is rarer than it sounds**, because somebody almost always needs to adjust the result later.

Those two questions eliminate most of the table, and both are **questions about your situation rather than about the algorithms** — which is why method selection is not a technical comparison exercise.

Then the third question is the budget. Microseconds per agent per frame (Module 11 §1) rules out most of what remains — which is why **precomputation keeps appearing as the answer** throughout this course (pattern databases, hierarchies, flow fields, offline-solved MDPs). **If the budget is microseconds and the computation is milliseconds, move the computation offline**, and that single move resolves a remarkable number of apparent impossibilities.

2 Symbolic and learned

	This course's methods	Learned methods
Guarantees	Optimality, completeness, and machine-checkable proofs (Modules 03, 08).	None.
Debugging	Trace the decision — the costs, the heuristic values, the active tree path.	A saliency map and hope (CSCE 753 Module 08 §4).
Data required	None.	A great deal, and matched to the deployment distribution (CSCE 753 Module 12).
Perception	Cannot do it at all — these methods need symbols as input.	The only option (CSCE 753).
Rules too messy to state	Must be stated, or they are not in the system.	Learned from examples, which is the whole advantage.
Runtime cost	Microseconds on a CPU core.	Milliseconds, plus a GPU.

The division is clean when stated plainly: symbolic methods for crisp problems with known rules; learned methods for perception and for rules nobody can write down. Arguing for either side in general is a category error, because they are good at disjoint things — and the useful skill is recognising which part of your problem is which.

EVERY WORKING SYSTEM IS A HYBRID

A self-driving stack is learned perception (CSCE 753) feeding symbolic planning and control — and the division falls exactly at the interface between 'what is out there' and 'what to do', which is the division the table above predicts.

AlphaZero is MCTS with a learned evaluation function and prior (Module 05 §3) — a classical search with the part nobody could write down supplied by learning.

A game's AI is behaviour trees and pathfinding, with learned animation blending, and occasionally one learned component for a specific hard judgement that resisted authoring.

The pattern is consistent across all three: learning supplies what cannot be written down; structure supplies the guarantees, the speed, and the control. Neither replaces the other, and the interface between them is where the engineering is — specifically, the interface must carry uncertainty, because the learned side is sometimes wrong and the symbolic side will otherwise act on it with full confidence (CSCE 753 Module 13 §4's abstention argument, from the consuming side).

3 Evaluating an agent

- **A trivial baseline. Random action, greedy one-step, or do-nothing** — and **report it**. CSCE 633 Module 02's discipline, unchanged across eight courses, and it catches the case where your elaborate agent is worse than moving toward the player.
- **Nodes expanded, or decisions per second**, not only wall-clock time (Module 03 §4) — so the measurement separates the algorithm from the implementation and can be compared against anything.
- **p99 latency, not the mean. A frame budget is a hard deadline and the mean does not describe compliance with it** — one search in a hundred overrunning is a visible stutter twice a second at 60 Hz (CSCE 678 Module 06, Module 04 §4).
- **Behaviour on cases you did not design for**. The agent will meet them — a player will stand somewhere unexpected, a level will have a geometry you did not test — and **what it does then is a property of the system you should know rather than discover**.
- **And a failure list. The situations where it does something wrong, with the identified cause of each** — which is the deliverable, exactly as it was in CSCE 753 Module 12 §4 and CSCE 633 Module 13. **For a game agent, add a human judgement: 'is it interesting to play against' is the real metric and no computation measures it**, so the evaluation must include someone playing it.

4 Claiming honestly

WHAT AN AI SYSTEM CAN HONESTLY PROMISE

'A* returns the shortest path on this navigation mesh, provably, in under 200 microseconds at the 99th percentile with 100 active agents.' Every clause is checkable, and the guarantee is a real one because the method has one (Module 03 §1).

'The planner finds a plan whenever one exists within 8 actions; beyond that it times out and the agent falls back to the behaviour tree.' The limit stated, and the fallback stated — which is what makes it a usable claim rather than an optimistic one.

'The policy is optimal for this MDP. The MDP assumes the player moves independently of the agent, which is false.' *Optimal for this model* — CSCE 669 Module 13 §4's four words, and **the model's falsehood stated in the same breath** (Module 01 §2's deliberate lie, written down where someone can read it).

And what you cannot honestly say: 'the agent is intelligent'. It searches a space you defined, with a heuristic you chose, under a model you approximated, within a budget you set. Say that instead — it is more impressive, it is checkable, and it is true.

WHERE THIS COURSE LEAVES YOU

You can formulate a problem as a state space, choose a search strategy from its properties, design and verify a heuristic, and ship pathfinding that works at production scale — which is the most immediately useful skill in the course for this track.

You can model a problem as constraints, plan with declarative actions and automatically derived heuristics, use a SAT or SMT solver rather than writing a custom search, reason probabilistically about hidden state, and solve a Markov decision process.

And you can choose an agent architecture and defend the choice against the alternatives — which is the thing this track needed, because **every game engine contains most of this course**, and the engineering question is almost never 'which algorithm' but 'which layer, at which rate, within which budget'.

The closing rule is the program's, unchanged across twenty courses: state what you measured, state what you assumed, and never claim more than you established. In artificial intelligence it means saying which space you searched — because the space was your approximation of the world (Module 01 §2), and the agent is exactly as good as that approximation and no better.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Russell & Norvig — AIMA, chapter 1 and the concluding chapter	The field's own account of the symbolic/learned division of §2 , from a source that covers both.
Game AI Pro — the architecture and postmortem chapters (free)	§1's table as practitioners apply it , with the constraints that drove each decision stated.
Sculley et al. — Hidden Technical Debt in Machine Learning Systems (free)	Why the hybrid interface of §2 is where the maintenance cost accumulates — relevant well beyond machine learning.
Berkeley CS188 — the full project set (free)	If you want more practice , the five Pacman projects cover Modules 02, 03, 05, 06, 09, and 10 with automatic grading.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Take three problems from your own work** and place each on §1's table, with the reasoning written out.
- 2 **Answer the two questions of §1's callout** for each, and report whether they changed your choice.
- 3 **Compute the budget** for each and check whether the chosen method fits.

- 4 **For one problem, identify which part is symbolic and which is learned**, and describe the interface between them.
- 5 **Specify what that interface must carry**, including the uncertainty.
- 6 **Implement a trivial baseline** for your project agent and report how it compares.
- 7 **Measure p50 and p99 decision latency** and report both.
- 8 **Construct five situations you did not design for** and report the agent's behaviour in each.
- 9 **Write the failure list** with a cause for each entry.
- 10 **Project 2 is now due.** Submit the agent, the architecture justification against two alternatives, the baseline comparison, the p99 latency, and the honest statement of what it cannot do.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Give the problem-to-method table from memory.
- 2 What two questions resolve most method choices?
- 3 Why is precomputation the recurring answer?
- 4 Compare symbolic and learned methods on six axes.
- 5 Why is arguing for either side in general a category error?
- 6 Give three hybrid systems and say where the division falls in each.
- 7 What must the interface between learned and symbolic parts carry?
- 8 Name five components of an agent evaluation.
- 9 Give three honest claims and the one thing you cannot say.
- 10 In this course, what does 'state what you assumed' specifically mean?